Tutoriumsblatt 14

Projektwerkstatt zu zweit

Bearbeitungszeit etwa 90 Minuten · 40 Punkte · Lösungen im Anhang ab Seite 5

Nach diesem Blatt kannst du

- Retrieval mit Recall@k und MRR vergleichen und RAG-Fehler eingrenzen,
- gepaarte Evals mit Wilson-Intervall und McNemar-Test ehrlich auswerten,
- Kosten pro Erfolg berechnen und Entscheidungen mit Messung protokollieren,
- Grenzen des Agent-Loops verfolgen und das Risiko der Demo abschätzen,
- Schutzschichten begründen und Zahlen für das Abnahmegespräch formulieren.

So arbeitest du mit dem Blatt

Wiederholung der Module 2 bis 13 wie zur Klausurvorbereitung, verankert in eurem Abschlussprojekt: Person A und Person B führen ihre Bausteine zum Instandhaltungs-Assistenten der Nordwerk GmbH (Track B) zusammen. Alle Zahlen sind ausgedachte Beispielwerte. Aufgaben 3 und 5 nutzen Ergebnisse aus Aufgabe 2, Aufgabe 4 das Kostenmodell aus Aufgabe 3. Erlaubt sind Taschenrechner und Formelblätter. Rechne mit drei Nachkommastellen. Bezug zur Vorlage `W14_Projekt_Demo.ipynb`: Aufgabe 4 und 5 (a) gehören zu Schritt 2 und 3, Aufgaben 2, 3 und 5 zu Schritt 4 und 5. Rechnet erst allein und erklärt euch dann gegenseitig die Lösungen.

Aufgabe 1 Retrieval messen, RAG-Fehler eingrenzen

7 Punkte · etwa 15 Min.

Bezug: Modul 7, Lerneinheiten 2, 5, 6 und 7 (Chunking, BM25, Recall@k und MRR, Fehlersuche)

Person A hat feste Chunks (F) gebaut, Person B Struktur-Chunks mit Überschriftenpfad (S). Die Tabelle nennt für acht gemeinsame Testfragen den Rang des ersten relevanten Chunks („keiner“: nicht unter den ersten zehn). Frage 3 fragt nach einem Tabellenwert, Frage 5 nach dem Fehlercode E-217.

Testfrage	1	2	3	4	5	6	7	8
F (feste Chunks)	1	3	keiner	2	6	1	4	1
S (Struktur-Chunks)	1	1	5	2	keiner	1	2	3

(a) Berechne für beide Varianten Recall@1, Recall@3, Recall@5 und den MRR. [3 P]

(b) Das Tool `search_knowledge_base` eures zusammengeführten Servers legt bisher die besten $k = 3$ Abschnitte in den Prompt. Welche Variante nehmt ihr, und wie belastbar ist der Unterschied? Welche zwei Testfragen schaut ihr euch einzeln an, und welche Erklärung aus Modul 7 liegt jeweils nahe? [2 P]

(c) Ein Testfall scheitert. Der Trace zeigt (gekürzt):

```
Frage:       Wie oft muss das Spindellager nachgeschmiert werden?
Referenz:    alle 500 Betriebsstunden (Handbuch, Abschnitt 4.2)
Aktion:      search_knowledge_base({"query": "Spindellager nachschmieren"})
Beobachtung: [1] 4.5 Hauptantrieb: Keilriemen alle 1000 Betriebsstunden prüfen
             [2] 4.1 Spindel: zulässige Drehzahlen und Schmierstoff
             [3] 6.3 Kühlmittel: Konzentration wöchentlich prüfen
Antwort:     Das Spindellager wird alle 1000 Betriebsstunden nachgeschmiert.
Nachträglich mit k = 10 gesucht: Abschnitt 4.2 steht auf Rang 4.
```

Geh die vier Stufen der Fehlersuche durch (Chunk vorhanden, Rang, im Prompt, vom Modell genutzt). Wo liegt der Fehler, und was hat das Modell zusätzlich falsch gemacht? Nenne zwei Maßnahmen und sag, wie ihr ihre Wirkung prüft. [2 P]

Aufgabe 2 Gepaart vergleichen: Wilson und McNemar

8 Punkte · etwa 18 Min.

Bezug: Modul 9, Lerneinheiten 5 bis 7 (Wilson, McNemar, Fehlschlüsse); Modul 14, Lerneinheiten 1 und 4

Person A hat einen ReAct-Agenten ohne Plan gebaut (V1), Person B einen Agenten mit Plan-then-Execute (V2). Beide laufen auf eurem gemeinsamen Testset mit $n = 30$ Fällen (12 normal, 8 Rand, 6 negativ, 4 Agent). V1 löst 21 Fälle, V2 löst 25 Fälle, und 19 Fälle lösen beide.

(a) Stelle die Vierfeldertafel auf (a beide richtig, b nur V1, c nur V2, d beide falsch). [1 P]

(b) Berechne das 95-%-Wilson-Intervall für V2 Schritt für Schritt: $\hat{p}$, z^2/n, Nenner, Mitte, halbe Breite und Grenzen ($z = 1{,}96$). Zur Kontrolle: V1 (21 von 30) ergibt etwa 52 bis 83 %. [3 P]

(c) Berechne den p-Wert des exakten McNemar-Tests $p = \min\left(1,\ 2\sum_{i=0}^{\min(b,c)} \binom{b+c}{i} 0{,}5^{b+c}\right)$. [2 P]

(d) Person B sagt: „V2 ist 13 Prozentpunkte besser, das ist eindeutig.“ Person A sagt: „Die Intervalle überlappen, also sind beide gleich gut.“ Beurteile beide Aussagen. Zeige: Selbst wenn alle Wechsel zugunsten von V2 ausfallen ($b = 0$), braucht der exakte Test mindestens sechs Wechsel für $p < 0{,}05$. Was tut ihr mit den b Fällen? [2 P]

Aufgabe 3 Kosten pro Erfolg und Entscheidung

8 Punkte · etwa 17 Min.

Bezug: Modul 10, Lerneinheit 3; Modul 13, Lerneinheit 6 (Kosten pro Erfolg); Modul 14, Lerneinheiten 1 und 4

Modell: Jeder Modellaufruf schickt den festen Anfang $T_0 = 2500$ Tokens (System-Prompt, Tools, Frage) und den Verlauf, der pro Aufruf um $\Delta = 600$ Tokens wächst, und erzeugt im Mittel 120 Output-Tokens. Eine Aufgabe braucht mit V1 im Mittel 4 Aufrufe, mit V2 im Mittel 5 (inklusive Planungsaufruf). Referenz: Claude Haiku 4.5 mit 1 und 5 US-Dollar pro Million Input- bzw. Output-Tokens (Stand 04.10.2026).

(a) Berechne mit $S(n) = n\,T_0 + \frac{n(n-1)}{2}\,\Delta$ die Input-Tokens pro Aufgabe für V1 und V2 und die Output-Tokens pro Aufgabe. [2 P]

(b) Berechne die Referenzkosten eines Versuchs in Cent für V1 und V2. Zur Kontrolle: V1 kostet 1,60 Cent. [2 P]

(c) Berechne mit den Erfolgsquoten aus Aufgabe 2 die Kosten und die Tokens pro erfolgreicher Aufgabe, $\bar{K}_{\text{Erfolg}} = \bar{K} \cdot n/k$. Warum ist das die richtige Kennzahl für den Vergleich? [2 P]

(d) Schreibe den Eintrag für euer Entscheidungsprotokoll im Format aus der Lerneinheit (Überschrift mit Entscheidung, Datum, Varianten, Messung, Grund). Nutze die Zahlen aus Aufgabe 2 und 3. [2 P]

Aufgabe 4 Agent-Loop und Demo: Zuverlässigkeit

9 Punkte · etwa 20 Min.

Bezug: Modul 10, Lerneinheit 3; Modul 11, Lerneinheit 1; Modul 13, Lerneinheit 5; Modul 14, Lerneinheiten 2, 3

In der Demo bleibt ein Gespräch offen, bis jemand „Neues Gespräch“ drückt. Der Verlauf aller Fragen wächst also weiter. Die Vorlage prüft ihre Grenzen so (stark vereinfacht):

```
MAX_SCHRITTE = 8            # Modellaufrufe pro Frage
BUDGET_TOKENS = 150_000     # Input-Tokens pro Gespräch
def laufe(s):               # läuft einmal pro Frage
    schritte = 0
    while True:
        if schritte >= MAX_SCHRITTE or s["tokens"]["input"] > BUDGET_TOKENS:
            return "limit"
        a = modell_aufrufen(s["nachrichten"])   # ganzer Verlauf
        schritte += 1
        s["tokens"]["input"] += a.usage.prompt_tokens
        if not a.tool_calls:
            return "antwort"
        ...                                     # Tools ausführen
```

(a) Nimm das Modell aus Aufgabe 3 für alle Aufrufe des Gesprächs an (Aufruf k schickt $T_0 + (k-1)\Delta$), jede Frage braucht 4 Aufrufe. Vor welchem Aufruf greift das Budget, in welcher Frage? Wie viele Input-Tokens sind dann verbraucht, und was kosten sie kommerziell? Warum wird das Budget um höchstens einen Aufruf überschritten? Zur Kontrolle: $S(18) = 136\,800$. [3 P]

(b) Drei Demo-Fragen: Der Normalfall gelingt mit 0,95, die mehrschrittige Frage braucht vier Tool-Schritte mit je 0,95, die schreibende Aktion mit Bestätigung gelingt mit 0,90. Wie wahrscheinlich läuft die Demo bei Unabhängigkeit fehlerfrei durch? Was ändert eine mehrschrittige Frage mit nur zwei Schritten? [2 P]

(c) Herleiten: Die Lerneinheit nennt für $k = n$ Erfolge die Wilson-Untergrenze $1 - \frac{z^2}{n+z^2}$. Leite sie aus der Wilson-Formel her, so wie Modul 13 die Obergrenze für $k = 0$ hergeleitet hat. [2 P]

(d) Ihr habt die mehrschrittige Frage zehnmal geprobt, und sie hat zehnmal geklappt. Wo liegt die Wilson-Untergrenze? Wie viele fehlerfreie Proben bräuchtet ihr für eine Untergrenze von mindestens 90 %? Was folgt daraus für euer Demo-Skript? [2 P]

Aufgabe 5 Sicherheit und Zahlen für die Abnahme

8 Punkte · etwa 18 Min.

Bezug: Modul 8, Lerneinheit 5; Modul 13, Lerneinheiten 3 bis 5; Modul 14, Lerneinheiten 3 bis 5

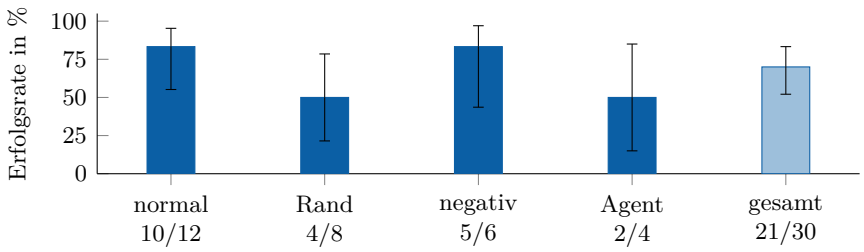
Euer gemeinsamer `team_server.py` bietet fünf Tools: `search_knowledge_base` liefert Abschnitte aus dem Handbuch der Maschinen. `list_tickets` listet Störungsmeldungen, frei formuliert von Mitarbeitenden, mit Name und Rufnummer der meldenden Person. `get_sensor_data` liefert Mess- und Grenzwerte aus der Maschinensteuerung. `create_ticket` legt ein Störungsticket an, und `send_report` schickt einen Bericht an eine beliebige E-Mail-Adresse.

(a) Welche Tools gehören in `SCHREIBENDE_TOOLS`, welche in `FREMDE_QUELLEN`? Begründe, auch für `get_sensor_data`. Was macht die Vorlage mit Ergebnissen fremder Quellen? [2 P]

(b) Zeige, dass diese Tools die gefährliche Kombination aus Modul 13 enthalten, und benenne ihre drei Ecken. Welche Maßnahme senkt das Risiko eines Datenabflusses am zuverlässigsten? Warum reicht ein Satz im System-Prompt nicht? [2 P]

(c) Red-Teaming über Kreuz: (i) Eine präparierte Meldung lässt den Agenten ein Not-Aus-Ticket mit Priorität hoch für M-03 vorschlagen. (ii) Ein präparierter Handbuch-Abschnitt verlangt, die Ticketliste an eine externe Adresse zu schicken. Eingebaut sind Kennzeichnung, Prüf-Call, Bestätigung durch eine Person und eine Allowlist, die `send_report` nur an interne Adressen zulässt. Ausgedachte Werte: Mit Kennzeichnung schlägt das Modell den Angriff in 25 % der Fälle vor, der Prüf-Call lässt 20 % durch, die aufmerksame Person 10 %. Schätze $P(\text{Schaden})$ für (i) und (ii) mit dem Schweizer-Käse-Modell und nenne die Annahme, die am ehesten verletzt ist. Danach kamen von 15 Angriffen 0 durch. Bis wohin reicht das 95-%-Wilson-Intervall? [2 P]

(d) Das Diagramm zeigt den finalen Eval-Lauf von V1 mit 95-%-Wilson-Intervallen (Beispiel). Welche Fallart nennt ihr als Schwäche, und warum sind Aussagen pro Fallart nur Hinweise? Formuliere für das Abnahmegespräch zwei Sätze: einen zum Eval mit Aufwand, einen zur Sicherheit. [2 P]



Aufgabe 6 (⋆) Prompt Caching im Demo-Gespräch

Zusatz · etwa 10 Min.

Bezug: Wiederholung Modul 11, Lerneinheit 7 (Prompt Caching)

Von den $T_0 = 2500$ Tokens jedes Aufrufs sind $P = 2000$ ein fester Präfix aus System-Prompt und Tools. Schreiben in den Cache kostet das 1,25-Fache, Lesen das 0,1-Fache des Input-Preises (Haiku 4.5: 1 US-Dollar pro Million).

(a) Die Vorlage schreibt „Heute ist der 09.10.2026“ an den Anfang des System-Prompts. Schadet das dem Caching im Gespräch? Und eine Uhrzeit mit Sekunden?

(b) Berechne die Kosten des Präfixes über die 19 Aufrufe aus Aufgabe 4 ohne und mit Caching, bestätige $E(19) = 0{,}9 - 1{,}15/19$ und gib an, um wie viel Prozent der gesamte Input des Gesprächs sinkt.

(c) Warum verträgt sich Kompaktierung schlecht mit Caching? Was spart Caching in der KI-Toolbox?

Anhang: Lösungen

Erst selbst rechnen, dann vergleichen. Die Lösungen zeigen den Rechenweg und typische Fehler.

Lösung zu Aufgabe 1 Retrieval messen, RAG-Fehler eingrenzen

(a) Recall@k zählt die Fragen mit Rang $\leq k$, der MRR mittelt 1/Rang, ohne Treffer zählt 0.

Variante	Recall@1	Recall@3	Recall@5	MRR
F	$3/8 = 37{,}5\,\%$	$5/8 = 62{,}5\,\%$	$6/8 = 75\,\%$	0,531
S	$3/8 = 37{,}5\,\%$	$6/8 = 75\,\%$	$7/8 = 87{,}5\,\%$	0,567

Die MRR-Werte im Einzelnen:

$$\mathrm{MRR}_F = \tfrac{1}{8}\left(1 + \tfrac{1}{3} + 0 + \tfrac{1}{2} + \tfrac{1}{6} + 1 + \tfrac{1}{4} + 1\right) = \tfrac{4{,}25}{8} \approx 0{,}531,$$
$$\mathrm{MRR}_S = \tfrac{1}{8}\left(1 + 1 + \tfrac{1}{5} + \tfrac{1}{2} + 0 + 1 + \tfrac{1}{2} + \tfrac{1}{3}\right) \approx \tfrac{4{,}533}{8} \approx 0{,}567.$$

(b) Bei $k = 3$ zählt Recall@3: S liegt mit 6 gegen 5 von 8 vorn, auch der MRR ist etwas höher. Belastbar ist das nicht. Bei $k = 3$ unterscheiden sich die Varianten nur in Frage 7 (Rang 4 gegen 2), eine Frage macht $1/8 = 12{,}5$ Prozentpunkte aus, und die Wilson-Intervalle für Recall@3 reichen von 31 bis 86 % (5 von 8) und von 41 bis 93 % (6 von 8). Schaut euch die Fragen einzeln an, die je eine Variante ganz verfehlt: **Frage 3** (Tabellenwert) findet F gar nicht, denn feste Chunks zerschneiden Tabellen. S findet sie auf Rang 5, weil Struktur-Chunks Tabellen zusammenhalten. **Frage 5** (Fehlercode E-217) findet S gar nicht, denn Kennungen findet eine Embedding-Suche schlecht, BM25 oder eine hybride Suche mit Reciprocal Rank Fusion dagegen gut. Musterentscheidung: S nehmen, weil sie den Tabellenfall überhaupt findet (mit $k = 5$ oder Reranking käme er in den Prompt), eine hybride Suche für Kennungen prüfen und beides mit Grund ins Protokoll schreiben. Eine Entscheidung für F gibt ebenfalls volle Punkte, wenn sie die fehlende Belastbarkeit nennt und begründet ist.

(c) Die vier Stufen:

1. Chunk vorhanden: ja, Abschnitt 4.2 steht im Index, die Suche mit $k = 10$ findet ihn.
2. Rang: 4, also knapp hinter der Grenze.
3. Im Prompt: nein, das Tool liefert nur die ersten $k = 3$ Abschnitte. **Hier liegt der Fehler.**
4. Vom Modell genutzt: Das Modell hat zusätzlich die 1000 Betriebsstunden des Keilriemens auf das Spindellager übertragen. Diese Aussage deckt keine Beobachtung. Richtig wäre: „Die gefundenen Abschnitte nennen kein Intervall für das Spindellager.“

Maßnahmen, zwei genügen: k auf 5 erhöhen (kostet Input-Tokens in jeder folgenden Runde), Reranking mit einem Cross-Encoder auf mehr Kandidaten, Struktur-Chunks oder hybride Suche, im System-Prompt die Regel „nur Belegtes, sonst sagen, dass es fehlt“ schärfen. Wirkung prüfen: Recall@k und MRR auf den Testfragen neu messen, den Fall mit Pflichtbegriff „500“ als Testfall aufnehmen und danach das ganze Eval laufen lassen, um Regressionen zu sehen.

Typischer Fehler

Den Rang ab 0 zählen wie in Python, oder die Frage ohne Treffer aus dem MRR weglassen. Dann steigt der MRR von F fälschlich auf $4{,}25/7 \approx 0{,}607$. Bei der Fehlersuche wird oft das Modell beschuldigt, obwohl der richtige Chunk den Prompt nie erreicht hat.

Bewertung: (a) Recall-Werte beider Varianten 2 P (je Zeile 1 P), MRR beider Varianten 1 P. (b) Belastbarkeit mit Größenordnung einer Frage 1 P, Fragen 3 und 5 mit Erklärung 1 P. (c) richtige Stufe mit Begründung 1 P, zusätzlicher Modellfehler und zwei Maßnahmen mit Prüfung 1 P.

Lösung zu Aufgabe 2 Gepaart vergleichen: Wilson und McNemar

(a) $a = 19$, $b = 21 - 19 = 2$, $c = 25 - 19 = 6$, $d = 30 - 19 - 2 - 6 = 3$.

	V2 richtig	V2 falsch
V1 richtig	$a = 19$	$b = 2$
V1 falsch	$c = 6$	$d = 3$

(b) $\hat{p} = 25/30 \approx 0{,}833$, $z^2/n = 3{,}8416/30 \approx 0{,}1281$, Nenner $1 + 0{,}1281 = 1{,}1281$.

$$\text{Mitte} = \frac{\hat{p} + \frac{z^2}{2n}}{1 + \frac{z^2}{n}} = \frac{0{,}8333 + 0{,}0640}{1{,}1281} \approx 0{,}795,$$

$$\text{halbe Breite} = \frac{z\sqrt{\frac{\hat{p}(1-\hat{p})}{n} + \frac{z^2}{4n^2}}}{1 + \frac{z^2}{n}} = \frac{1{,}96\sqrt{0{,}00463 + 0{,}00107}}{1{,}1281} = \frac{1{,}96 \cdot 0{,}0755}{1{,}1281} \approx 0{,}131.$$

Grenzen: $0{,}795 \pm 0{,}131$, also $[0{,}664;\ 0{,}927]$, **66 bis 93 %**. Die Mitte liegt unter $\hat{p}$, Wilson zieht zur Mitte.

(c) $b + c = 8$ Wechsel, $\min(b, c) = 2$:

$$p = 2 \cdot \frac{\binom{8}{0} + \binom{8}{1} + \binom{8}{2}}{2^8} = 2 \cdot \frac{1 + 8 + 28}{256} = \frac{74}{256} \approx \mathbf{0{,}289}.$$

(d) Person B: Die 13 Prozentpunkte ($25 - 21 = 4$ Fälle von 30) stammen aus 8 Wechseln. Eine mindestens so ungleiche Aufteilung tritt auch bei gleich guten Versionen in fast 29 % der Fälle auf. Eindeutig ist also nichts, die Verbesserung ist *nicht belegt*. Person A: Überlappende Intervalle (52 bis 83 % und 66 bis 93 %) sind ein grobes Kriterium und kein Beweis für Gleichheit. Fehlender Nachweis ist kein Nachweis der Gleichheit. Richtig ist der gepaarte Test, und der sagt: Die Daten können keinen Unterschied belegen.

Mit $b = 0$ und $c = m$ ist $p = 2 \cdot 0{,}5^m$. Für $m = 5$: $2/32 = 0{,}0625 > 0{,}05$, für $m = 6$: $2/64 \approx 0{,}031 < 0{,}05$. Also braucht der Test **mindestens sechs Wechsel**, und bei 30 Fällen mit wenigen Wechseln kann er kleine Unterschiede gar nicht nachweisen. Die $b = 2$ Fälle sind Regressionen von V2: Ihr schaut sie euch einzeln im Trace an, bevor ihr V2 übernehmt.

Typischer Fehler

Beim McNemar-Test nur den Summanden für $i = 0$ nehmen oder die Zwei für die zweite Seite vergessen. Häufig ist auch, zwei unabhängige Intervalle zu vergleichen, obwohl beide Versionen auf denselben Fällen liefen.

Bewertung: (a) 1 P. (b) $\hat{p}$, z^2/n und Nenner 1 P, Mitte 1 P, halbe Breite und Grenzen 1 P. (c) Ansatz mit $b + c = 8$ und Summe bis 2 1 P, Ergebnis 1 P. (d) beide Aussagen beurteilt 1 P, sechs Wechsel gezeigt und Umgang mit Regressionen 1 P. Folgefehler aus (a) werden nicht doppelt abgezogen.

Lösung zu Aufgabe 3 Kosten pro Erfolg und Entscheidung

(a) Aufruf k schickt $T_0 + (k-1)\Delta$: 2500, 3100, 3700, 4300 und bei V2 noch 4900 Tokens.

$S(4) = 4 \cdot 2500 + \frac{4 \cdot 3}{2} \cdot 600 = 10\,000 + 3\,600 = \mathbf{13\,600}$, $S(5) = 12\,500 + 10 \cdot 600 = \mathbf{18\,500}$.

Output: V1 $4 \cdot 120 = 480$, V2 $5 \cdot 120 = 600$ Tokens. Zusammen 14 080 bzw. 19 100 Tokens pro Aufgabe.

(b) V1: $(13\,600 \cdot 1 + 480 \cdot 5)/10^6 = 16\,000/10^6 = 0{,}016$ USD = **1,60** Cent. V2: $(18\,500 + 3\,000)/10^6 = 0{,}0215$ USD = **2,15** Cent. Ein Versuch mit Plan kostet also 34 % mehr.

(c) V1: $1{,}60 \cdot 30/21 \approx$ **2,29** Cent und $14\,080 \cdot 30/21 \approx 20\,114$ Tokens pro Erfolg. V2: $2{,}15 \cdot 30/25 =$ **2,58** Cent und $19\,100 \cdot 1{,}2 = 22\,920$ Tokens pro Erfolg. Die Kennzahl ist richtig, weil gescheiterte Versuche ebenfalls bezahlt werden. Wer bis zum Erfolg wiederholt, braucht im Mittel $1/p$ Versuche (Modul 13). Der Vergleich pro Versuch würde die höhere Erfolgsrate von V2 ignorieren. Hier bleibt V2 auch pro Erfolg 13 % teurer. Beachte: Die Erfolgsquoten sind selbst unsicher (Intervalle aus Aufgabe 2), die Kosten pro Erfolg also auch.

(d) Musterlösung:

```
## Planungsschritt: nein (V1, ReAct ohne Plan)
Datum:     Tag des finalen Vergleichs
Varianten: V1 ReAct ohne Plan (Person A), V2 Plan-then-Execute (Person B)
Messung:   gemeinsames Testset, 30 Fälle, gepaart: V1 21, V2 25 richtig,
           b = 2, c = 6, exakter McNemar p = 0,29: kein belegbarer Unterschied.
           Pro Aufgabe 14080 gegen 19100 Tokens, pro Erfolg 2,29 gegen 2,58 Cent.
Grund:     ohne Plan weniger Tokens und Kosten pro Erfolg, einfacher zu erklären.
           Die 2 Regressionen und 6 Gewinne von V2 im Trace angesehen.
           Planung als Ausblick notiert.
```

Auch „Planungsschritt: ja“ gibt volle Punkte, wenn der Eintrag den p-Wert ehrlich nennt, die Mehrkosten angibt und einen sachlichen Grund liefert, etwa dass die sechs gewonnenen Fälle genau die mehrschrittigen Aufgaben der Zielgruppe sind.

> **Typischer Fehler**
> Mit $n(n+1)/2$ statt $n(n-1)/2$ rechnen: Beim ersten Aufruf ist der Verlauf noch leer. Bei den Kosten pro Erfolg wird oft mit der Erfolgsquote multipliziert statt durch sie geteilt. Das ergibt eine sinnlose, kleinere Zahl.

Bewertung: (a) $S(4)$ und $S(5)$ 1 P, Output-Tokens 1 P. (b) je Variante 1 P. (c) beide Kennzahlen 1 P, Begründung 1 P. (d) vollständiges Format mit Messung 1 P, nachvollziehbarer Grund 1 P.

Lösung zu Aufgabe 4 Agent-Loop und Demo: Zuverlässigkeit

(a) Gesprächsweit gilt $S(18) = 18 \cdot 2500 + \frac{18 \cdot 17}{2} \cdot 600 = 45\,000 + 91\,800 = 136\,800$. Vor Aufruf 19 prüft die Vorlage $136\,800 > 150\,000$: falsch, also läuft Aufruf 19 mit $2500 + 18 \cdot 600 = 13\,300$ Tokens. Danach sind es **150 100**. Vor Aufruf 20 ist $150\,100 > 150\,000$ wahr: Status `"limit"`. Das Budget greift also **vor Aufruf 20**. Die Fragen belegen die Aufrufe 1 bis 4, 5 bis 8, 9 bis 12, 13 bis 16 und 17 bis 20. In **Frage 5** laufen noch drei Aufrufe, dann bricht der Agent mit dem Hinweis auf die Grenze ab, und ihr drückt „Neues Gespräch“. Kommerziell kosten die Input-Tokens $150\,100/10^6$ USD $\approx 15{,}0$ Cent, dazu kommen $19 \cdot 120 = 2280$ Output-Tokens für 1,14 Cent. Höchstens ein Aufruf zu viel: Die Prüfung steht *vor* jedem Aufruf und kennt nur die

bisher verbrauchten Tokens. Ein Aufruf startet, solange die Grenze noch nicht überschritten ist, und erst er kann die Summe darüber heben. Danach startet keiner mehr. `MAX_SCHRITTE` greift hier nicht, denn jede Frage braucht nur 4 von 8 Aufrufen.

(b) Mehrschrittige Frage: $0{,}95^4 \approx 0{,}815$. Demo: $0{,}95 \cdot 0{,}815 \cdot 0{,}90 \approx \mathbf{0{,}696}$, gut zwei Drittel. Ungefähr jede dritte Demo stolpert also an mindestens einer Frage. Mit zwei Schritten: $0{,}95^2 = 0{,}9025$ und $0{,}95 \cdot 0{,}9025 \cdot 0{,}90 \approx 0{,}772$. Jeder Schritt kostet Zuverlässigkeit, wie bei r^m in der Lerneinheit.

(c) Mit $k = n$ ist $\hat{p} = 1$ und $\hat{p}(1-\hat{p}) = 0$. Unter der Wurzel bleibt $\frac{z^2}{4n^2}$, die Wurzel ist $\frac{z}{2n}$, mal z ergibt sich $\frac{z^2}{2n}$. Die Untergrenze ist

$$\frac{1 + \frac{z^2}{2n} - \frac{z^2}{2n}}{1 + \frac{z^2}{n}} = \frac{1}{1 + \frac{z^2}{n}} = \frac{n}{n + z^2} = 1 - \frac{z^2}{n + z^2}.$$

Sie ist das Spiegelbild der Obergrenze $\frac{z^2}{n+z^2}$ für $k = 0$. Kontrolle mit dem Wert der Lerneinheit: $5/8{,}8416 \approx 0{,}57$.

(d) $10/(10 + 3{,}8416) = 10/13{,}8416 \approx \mathbf{0{,}722}$, also 72 %. Bedingung $\frac{n}{n+z^2} \geq 0{,}9 \iff 0{,}1\,n \geq 0{,}9\,z^2 \iff n \geq 9z^2 \approx 34{,}6$, also **35 fehlerfreie Proben** (Kontrolle: $35/38{,}84 \approx 0{,}901$, $34/37{,}84 \approx 0{,}898$). Folgerung: Proben allein belegen keine hohe Zuverlässigkeit. Ins Demo-Skript gehören ein Plan B (Video), nur drei Fragen, eine mehrschrittige Frage mit möglichst wenigen Schritten und kein wörtliches Ankündigen der Antwort.

> **Typischer Fehler**
> In (a) den Verlauf pro Frage neu beginnen lassen. In der Demo schickt aber jeder Aufruf den ganzen Verlauf des Gesprächs, deshalb wächst der Verbrauch über die Fragen hinweg quadratisch. In (d) wird aus „zehnmal geklappt“ oft „zu 100 % zuverlässig“ gemacht.

Bewertung: (a) Aufruf und Frage 1 P, Tokens und Kosten 1 P, Begründung der höchstens einen Überschreitung 1 P. (b) Demo-Wahrscheinlichkeit 1 P, Vergleich mit zwei Schritten 1 P. (c) Einsetzen von $\hat{p} = 1$ 1 P, Vereinfachen 1 P. (d) Untergrenze 72 % und 35 Proben 1 P, Folgerung 1 P.

Lösung zu Aufgabe 5 Sicherheit und Zahlen für die Abnahme

(a) Die beiden Mengen lauten:

```
SCHREIBENDE_TOOLS = {"create_ticket", "send_report"}
FREMDE_QUELLEN = {"search_knowledge_base", "list_tickets"}
```

Die schreibenden Tools verändern etwas außerhalb des Gesprächs, ein Ticket entsteht oder eine E-Mail geht hinaus. Die fremden Quellen liefern Text, den Dritte geschrieben haben und der präpariert sein kann, Handbuch-Abschnitte und frei formulierte Meldungen. `get_sensor_data` liefert Zahlen aus der eigenen Maschinensteuerung, keinen frei formulierten Text, und gehört deshalb in keine der beiden Mengen. Wer das Tool vorsichtshalber trotzdem kennzeichnet, bekommt volle Punkte, denn die Kennzeichnung schadet nicht. Die Vorlage entschärft in Ergebnissen aus `FREMDE_QUELLEN` die spitzen Klammern und packt sie in `<fremder_inhalt quelle="...">`-Markierungen. Der System-Prompt erklärt, dass darin Daten stehen und keine Anweisungen.

(b) Private Daten: Namen und Rufnummern in `list_tickets`. Fremde Inhalte: Meldungstexte und Handbuch-Abschnitte. Kanal nach außen: `send_report` an beliebige Adressen. Kommen alle drei zusammen, kann eine versteckte Anweisung private Daten hinausschicken lassen. Am zuverlässigsten ist es, eine Ecke zu entfernen oder streng zu begrenzen: `send_report` nur an interne Empfänger, geprüft im Code mit `re.fullmatch` gegen ein Muster wie `[a-z0-9._%+-]+@nordwerk\.example`. Alternativ gibt `list_tickets` gar keine Kontaktdaten heraus. Ein Satz im System-Prompt reicht nicht, weil das Modell System-Prompt, Frage und Tool-Ergebnisse als eine einzige Tokenfolge sieht. Eine Anweisung im fremden Inhalt kann den Satz überstimmen. Training senkt nur die Erfolgsrate, eine harte Trennung gibt es nicht.

(c) (i) Die Allowlist hilft nicht, denn Maschine und Priorität sind gültig: $P(\text{Schaden}) = 0{,}25 \cdot 0{,}2 \cdot 0{,}1 = \mathbf{0{,}005}$, etwa einer von 200 Angriffen. (ii) Die Allowlist lässt externe Empfänger nie durch, ihr Durchlass ist $f = 0$, also $P(\text{Schaden}) = 0$ für genau diese Angriffsart. Am ehesten verletzt ist die Unabhängigkeit: Prüf-Call und Agent nutzen dasselbe Modell. Der Prüf-Call sieht die präparierte Meldung zwar nicht, aber eine plausibel wirkende Aktion (etwa ein Ticket, wenn der Nutzer nach neuen Meldungen gefragt hat) täuscht Prüf-Call und Person zugleich. Außerdem klickt, wer viele Bestätigungen sieht, irgendwann ungelesen weiter. Für 0 von 15 ist die Obergrenze $z^2/(n+z^2) = 3{,}8416/18{,}8416 \approx 0{,}204$, also **bis etwa 20 %**.

(d) Am schwächsten sind Randfälle mit 4 von 8 und die Agenten-Aufgaben mit 2 von 4. Als Schwäche nennt ihr die Randfälle, sie haben mehr Fälle und eine erklärbare Ursache. Die Intervalle pro Fallart sind riesig (Rand 22 bis 78 %, Agent 15 bis 85 %), weil nur 4 bis 12 Fälle dahinterstehen. Musterformulierung:

„21 von 30 Testfällen sind erfolgreich (70 %, 95-%-Intervall 52 bis 83 %), bei etwa 14 100 Tokens pro Aufgabe, kommerziell etwa 1,6 Cent pro Versuch. Am schwächsten sind Randfälle mit ungenauen Maschinennamen (4 von 8).“

„Nach dem Einbau von Allowlist und Prüf-Call kam von 15 Angriffsversuchen keiner durch. Das ist noch mit einer Erfolgsrate bis etwa 20 % vereinbar, präparierte Meldungen bleiben unser wichtigstes Restrisiko.“

Typischer Fehler

„Unser Agent ist zu 70 % genau“ oder „unser Agent ist sicher“: Ohne Anzahl der Fälle und Intervall lässt sich keine der beiden Zahlen einordnen, und 0 von 15 ist kein Beleg für Sicherheit.

Bewertung: (a) beide Mengen mit Begründung 1 P, `get_sensor_data` und Kennzeichnung 1 P. (b) drei Ecken 1 P, Maßnahme und Grenze des Prompt-Satzes 1 P. (c) beide Schadenswahrscheinlichkeiten mit Annahme 1 P, Obergrenze 20 % 1 P. (d) Schwäche mit Begründung der Unsicherheit 1 P, zwei Sätze mit Anteil, Anzahl, Intervall, Aufwand und Sicherheitszahl 1 P.

Lösung zu Aufgabe 6 Prompt Caching im Demo-Gespräch

(a) Nein. Das Datum ändert sich nur einmal am Tag, innerhalb eines Gesprächs bleibt der Präfix exakt gleich. Der Cache lebt bei Anthropic ohnehin nur fünf Minuten nach dem letzten Treffer. Eine Uhrzeit mit Sekunden im ersten Satz würde sich dagegen bei jedem Aufruf ändern. Ab dieser Stelle müssten alle Keys und Values neu berechnet werden, wiederverwendbar wäre fast nichts. Regel: feste Teile nach vorne, wechselnde ans Ende.

(b) Ohne Cache: $19 \cdot 2000 = 38\,000$ Tokens, also 3,8 Cent. Mit Cache: $2000 \cdot 1{,}25 + 18 \cdot 2000 \cdot 0{,}1 = 2500 + 3600 = 6100$, also 0,61 Cent. Ersparnis $1 - 6100/38\,000 \approx 0{,}839$, und $0{,}9 - 1{,}15/19 \approx 0{,}839$ ✓. Der gesamte Input des Gesprächs sinkt von 150 100 auf $150\,100 - 38\,000 + 6100 = 118\,200$ Token-Äquivalente, also nur um etwa 21 %, weil der wachsende Verlauf den größten Teil ausmacht. Da der Verlauf nur hinten wächst, ließe er sich ebenfalls cachen.

(c) Kompaktierung ersetzt den Anfang des Verlaufs durch eine Zusammenfassung. Ab dieser Stelle ist der Präfix ein anderer, und der Cache ist dort wertlos. Deshalb feste Teile vorne lassen und nur mit hoher Schwelle, also selten, kompaktieren. In der KI-Toolbox zahlt ihr nichts. Caching kann dort höchstens Rechen- und Wartezeit sparen, und zwar beim Einlesen, nicht beim Erzeugen der Antwort.